Core Java Hands-On Exercises

1. Hello World Program

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

2. Simple Calculator

```
import java.util.Scanner;

public class Calculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number: ");
        double a = sc.nextDouble();
        System.out.print("Enter second number: ");
        double b = sc.nextDouble();
        System.out.print("Choose operation (+,-,*,/): ");
        char op = sc.next().charAt(0);

        double result;
        switch (op) {
            case '+': result = a + b; break;
            case '-': result = a - b; break;
            case '*': result = a * b; break;
            case '/': result = a / b; break;
            default:
                System.out.println("Invalid operation");
                return;
        }
        System.out.println("Result: " + result);
    }
}
```

3. Even or Odd Checker

```
import java.util.Scanner;
```

```
public class EvenOddChecker {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter an integer: ");
        int num = sc.nextInt();

        if (num % 2 == 0) {
            System.out.println(num + " is Even");
        } else {
            System.out.println(num + " is Odd");
        }
    }
}
```

4. Leap Year Checker

```
import java.util.Scanner;

public class LeapYearChecker {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a year: ");
        int year = sc.nextInt();

        if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
            System.out.println(year + " is a Leap Year");
        } else {
            System.out.println(year + " is not a Leap Year");
        }
    }
}
```

5. Multiplication Table

```
import java.util.Scanner;

public class MultiplicationTable {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int num = sc.nextInt();
```

```
        for (int i = 1; i <= 10; i++) {
            System.out.println(num + " x " + i + " = " + (num * i));
        }
    }
}
```

6. Data Type Demonstration

```
public class DataTypeDemo {
    public static void main(String[] args) {
        int age = 25;
        float height = 5.9f;
        double salary = 55000.75;
        char grade = 'A';
        boolean isEmployed = true;

        System.out.println("age: " + age);
        System.out.println("height: " + height);
        System.out.println("salary: " + salary);
        System.out.println("grade: " + grade);
        System.out.println("isEmployed: " + isEmployed);
    }
}
```

7. Type Casting Example

```
public class TypeCastingDemo {
    public static void main(String[] args) {
        double d = 9.78;
        int i = (int) d;
        System.out.println("double to int: " + i);

        int num = 10;
        double castedNum = (double) num;
        System.out.println("int to double: " + castedNum);
    }
}
```

8. Operator Precedence

```
public class OperatorPrecedence {
```

```
    public static void main(String[] args) {
        int result = 10 + 5 * 2;
        System.out.println("Result: " + result);

        int result2 = (10 + 5) * 2;
        System.out.println("Result2: " + result2);
    }
}
```

Output: Result = 20 (multiplication happens before addition), Result2 = 30 (parentheses evaluated first).

9. Grade Calculator

```
import java.util.Scanner;

public class GradeCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter marks: ");
        int marks = sc.nextInt();

        char grade;
        if (marks >= 90) grade = 'A';
        else if (marks >= 80) grade = 'B';
        else if (marks >= 70) grade = 'C';
        else if (marks >= 60) grade = 'D';
        else grade = 'F';

        System.out.println("Grade: " + grade);
    }
}
```

10. Number Guessing Game

```
import java.util.Random;
import java.util.Scanner;

public class NumberGuessingGame {
    public static void main(String[] args) {
        Random random = new Random();
        int target = random.nextInt(100) + 1;
```

```
        Scanner sc = new Scanner(System.in);
        int guess;

        do {
            System.out.print("Guess a number (1-100): ");
            guess = sc.nextInt();
            if (guess > target) {
                System.out.println("Too high");
            } else if (guess < target) {
                System.out.println("Too low");
            } else {
                System.out.println("Correct guess!");
            }
        } while (guess != target);
    }
}
```

11. Factorial Calculator

```
import java.util.Scanner;

public class FactorialCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a non-negative integer: ");
        int num = sc.nextInt();

        long factorial = 1;
        for (int i = 1; i <= num; i++) {
            factorial *= i;
        }
        System.out.println("Factorial: " + factorial);
    }
}
```

12. Method Overloading

```
public class MethodOverloadingDemo {

    static int add(int a, int b) {
        return a + b;
```

```
    }

    static double add(double a, double b) {
        return a + b;
    }

    static int add(int a, int b, int c) {
        return a + b + c;
    }

    public static void main(String[] args) {
        System.out.println(add(2, 3));
        System.out.println(add(2.5, 3.5));
        System.out.println(add(1, 2, 3));
    }
}
```

13. Recursive Fibonacci

```
import java.util.Scanner;

public class RecursiveFibonacci {

    static int fibonacci(int n) {
        if (n <= 1) return n;
        return fibonacci(n - 1) + fibonacci(n - 2);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();
        System.out.println("Fibonacci(" + n + ") = " + fibonacci(n));
    }
}
```

14. Array Sum and Average

```
import java.util.Scanner;

public class ArraySumAverage {
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        int sum = 0;

        for (int i = 0; i < n; i++) {
            System.out.print("Enter element " + (i + 1) + ": ");
            arr[i] = sc.nextInt();
            sum += arr[i];
        }

        double average = (double) sum / n;
        System.out.println("Sum: " + sum);
        System.out.println("Average: " + average);
    }
}
```

15. String Reversal

```
import java.util.Scanner;

public class StringReversal {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        String reversed = new StringBuilder(str).reverse().toString();
        System.out.println("Reversed: " + reversed);
    }
}
```

16. Palindrome Checker

```
import java.util.Scanner;

public class PalindromeChecker {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        String cleaned = str.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();
        String reversed = new StringBuilder(cleaned).reverse().toString();

        if (cleaned.equals(reversed)) {
            System.out.println("Palindrome");
        } else {
            System.out.println("Not a Palindrome");
        }
    }
}
```

17. Class and Object Creation

```
public class Car {
    String make;
    String model;
    int year;

    Car(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }

    void displayDetails() {
        System.out.println(year + " " + make + " " + model);
    }

    public static void main(String[] args) {
        Car car1 = new Car("Toyota", "Corolla", 2022);
        Car car2 = new Car("Honda", "Civic", 2023);
        car1.displayDetails();
        car2.displayDetails();
    }
}
```

18. Inheritance Example

```
class Animal {
    void makeSound() {
        System.out.println("Some generic sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Bark");
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Animal animal = new Animal();
        Dog dog = new Dog();
        animal.makeSound();
        dog.makeSound();
    }
}
```

19. Interface Implementation

```
interface Playable {
    void play();
}

class Guitar implements Playable {
    public void play() {
        System.out.println("Playing the guitar");
    }
}

class Piano implements Playable {
    public void play() {
        System.out.println("Playing the piano");
    }
}

public class InterfaceDemo {
```

```
    public static void main(String[] args) {
        Playable guitar = new Guitar();
        Playable piano = new Piano();
        guitar.play();
        piano.play();
    }
}
```

20. Try-Catch Example

```
import java.util.Scanner;

public class TryCatchDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter numerator: ");
        int a = sc.nextInt();
        System.out.print("Enter denominator: ");
        int b = sc.nextInt();

        try {
            int result = a / b;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero: " + e.getMessage());
        }
    }
}
```

21. Custom Exception

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class CustomExceptionDemo {

    static void checkAge(int age) throws InvalidAgeException {
        if (age < 18) {
```

```
            throw new InvalidAgeException("Age must be 18 or above");
        }
        System.out.println("Age is valid");
    }

    public static void main(String[] args) {
        try {
            checkAge(15);
        } catch (InvalidAgeException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

22. File Writing

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class FileWritingDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter text: ");
        String text = sc.nextLine();

        try (FileWriter writer = new FileWriter("output.txt")) {
            writer.write(text);
            System.out.println("Data written to output.txt");
        } catch (IOException e) {
            System.out.println("Error writing file: " + e.getMessage());
        }
    }
}
```

23. File Reading

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
```

```
public class FileReadingDemo {
   public static void main(String[] args) {
      try (BufferedReader reader = new BufferedReader(new
FileReader("output.txt"))) {
         String line;
         while ((line = reader.readLine()) != null) {
            System.out.println(line);
         }
      } catch (IOException e) {
         System.out.println("Error reading file: " + e.getMessage());
      }
   }
}
```

24. ArrayList Example

```
import java.util.ArrayList;
import java.util.Scanner;

public class ArrayListDemo {
   public static void main(String[] args) {
      ArrayList<String> names = new ArrayList<>();
      Scanner sc = new Scanner(System.in);

      System.out.print("How many names to add? ");
      int n = sc.nextInt();
      sc.nextLine();

      for (int i = 0; i < n; i++) {
         System.out.print("Enter name: ");
         names.add(sc.nextLine());
      }

      System.out.println("Student names: " + names);
   }
}
```

25. HashMap Example

```
import java.util.HashMap;
import java.util.Scanner;
```

```
public class HashMapDemo {
    public static void main(String[] args) {
        HashMap<Integer, String> students = new HashMap<>();
        Scanner sc = new Scanner(System.in);

        students.put(101, "Alice");
        students.put(102, "Bob");
        students.put(103, "Charlie");

        System.out.print("Enter student ID to lookup: ");
        int id = sc.nextInt();
        System.out.println("Name: " + students.get(id));
    }
}
```

26. Thread Creation

```
class MessageThread extends Thread {
    private String message;

    MessageThread(String message) {
        this.message = message;
    }

    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println(message + " - " + i);
        }
    }
}

public class ThreadDemo {
    public static void main(String[] args) {
        MessageThread t1 = new MessageThread("Thread 1");
        MessageThread t2 = new MessageThread("Thread 2");
        t1.start();
        t2.start();
    }
}
```

27. Lambda Expressions

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class LambdaDemo {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>(List.of("Charlie", "Alice", "Bob"));
        Collections.sort(names, (a, b) -> a.compareTo(b));
        System.out.println(names);
    }
}
```

28. Stream API

```
import java.util.List;
import java.util.stream.Collectors;

public class StreamDemo {
    public static void main(String[] args) {
        List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
        List<Integer> evenNumbers = numbers.stream()
                .filter(n -> n % 2 == 0)
                .collect(Collectors.toList());
        System.out.println(evenNumbers);
    }
}
```

29. Records

```
import java.util.List;
import java.util.stream.Collectors;

public record Person(String name, int age) {

    public static void main(String[] args) {
        Person p1 = new Person("Alice", 25);
        Person p2 = new Person("Bob", 17);
        System.out.println(p1);
        System.out.println(p2);
```

```
        List<Person> people = List.of(p1, p2);
        List<Person> adults = people.stream()
                .filter(p -> p.age() >= 18)
                .collect(Collectors.toList());
        System.out.println(adults);
    }
}
```

30. Pattern Matching for switch (Java 21)

```
public class PatternMatchingDemo {

    static String describe(Object obj) {
        return switch (obj) {
            case Integer i -> "Integer: " + i;
            case String s -> "String: " + s;
            case Double d -> "Double: " + d;
            default -> "Unknown type";
        };
    }

    public static void main(String[] args) {
        System.out.println(describe(10));
        System.out.println(describe("Hello"));
        System.out.println(describe(3.14));
    }
}
```

31. Basic JDBC Connection

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class JdbcConnectionDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/schooldb";
        String user = "root";
        String password = "root";
```

```
        try (Connection conn = DriverManager.getConnection(url, user, password);
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery("SELECT * FROM students")) {

            while (rs.next()) {
                System.out.println(rs.getInt("id") + " - " + rs.getString("name"));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

32. Insert and Update Operations in JDBC

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class StudentDAO {

    private Connection getConnection() throws Exception {
        return DriverManager.getConnection("jdbc:mysql://localhost:3306/schooldb",
"root", "root");
    }

    public void insertStudent(String name, int age) throws Exception {
        String sql = "INSERT INTO students (name, age) VALUES (?, ?)";
        try (Connection conn = getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, name);
            ps.setInt(2, age);
            ps.executeUpdate();
        }
    }

    public void updateStudent(int id, String name) throws Exception {
        String sql = "UPDATE students SET name = ? WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, name);
```

```
            ps.setInt(2, id);
            ps.executeUpdate();
        }
    }
}
```

33. Transaction Handling in JDBC

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class TransferService {

    public void transfer(int fromId, int toId, double amount) {
        String debitSql = "UPDATE accounts SET balance = balance - ? WHERE id
= ?";
        String creditSql = "UPDATE accounts SET balance = balance + ? WHERE id
= ?";

        try (Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/bankdb", "root", "root")) {

            conn.setAutoCommit(false);

            try (PreparedStatement debit = conn.prepareStatement(debitSql);
                 PreparedStatement credit = conn.prepareStatement(creditSql)) {

                debit.setDouble(1, amount);
                debit.setInt(2, fromId);
                debit.executeUpdate();

                credit.setDouble(1, amount);
                credit.setInt(2, toId);
                credit.executeUpdate();

                conn.commit();
                System.out.println("Transfer successful");
            } catch (Exception e) {
                conn.rollback();
                System.out.println("Transfer failed, rolled back");
```

```
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

34. Create and Use Java Modules

```
// module-info.java in com.utils module
module com.utils {
    exports com.utils;
}

// com/utils/StringUtils.java
package com.utils;
public class StringUtils {
    public static String greet(String name) {
        return "Hello, " + name;
    }
}
// module-info.java in com.greetings module
module com.greetings {
    requires com.utils;
}

// com/greetings/Main.java
package com.greetings;
import com.utils.StringUtils;

public class Main {
    public static void main(String[] args) {
        System.out.println(StringUtils.greet("World"));
    }
}
javac -d out --module-source-path src (src/com.utils/**, src/com.greetings/**)
java --module-path out -m com.greetings/com.greetings.Main
```

35. TCP Client-Server Chat

```
// Server.java
```

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) throws IOException {
        ServerSocket serverSocket = new ServerSocket(5000);
        System.out.println("Waiting for client...");
        Socket socket = serverSocket.accept();

        BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

        String message = in.readLine();
        System.out.println("Client: " + message);
        out.println("Message received");
    }
}
// Client.java
import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) throws IOException {
        Socket socket = new Socket("localhost", 5000);
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));

        out.println("Hello Server");
        System.out.println("Server: " + in.readLine());
    }
}
```

36. HTTP Client API (Java 11+)

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
```

```
public class HttpClientDemo {
   public static void main(String[] args) throws Exception {
      HttpClient client = HttpClient.newHttpClient();
      HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://api.github.com/users/octocat"))
            .GET()
            .build();

      HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());
      System.out.println("Status: " + response.statusCode());
      System.out.println("Body: " + response.body());
   }
}
```

37. Using javap to Inspect Bytecode

```
public class Sample {
   public int add(int a, int b) {
      return a + b;
   }
}
javac Sample.java
javap -c Sample
```

Output shows the compiled bytecode instructions for add() such as iload_1, iload_2, iadd, ireturn.

38. Decompile a Class File

```
public class Greeting {
   public static void main(String[] args) {
      System.out.println("Hello from Greeting class");
   }
}
javac Greeting.java
// Open Greeting.class in JD-GUI or CFR to view the decompiled source
```

39. Reflection in Java

```
import java.lang.reflect.Method;
```

```
public class ReflectionDemo {
    public static void main(String[] args) throws Exception {
        Class<?> clazz = Class.forName("java.lang.StringBuilder");
        Method[] methods = clazz.getDeclaredMethods();

        for (Method m : methods) {
            System.out.println(m.getName());
        }

        Object obj = clazz.getDeclaredConstructor().newInstance();
        Method append = clazz.getMethod("append", String.class);
        append.invoke(obj, "Hello Reflection");
        System.out.println(obj.toString());
    }
}
```

40. Virtual Threads (Java 21)

```
public class VirtualThreadDemo {
    public static void main(String[] args) throws InterruptedException {
        long start = System.currentTimeMillis();

        Thread[] threads = new Thread[100000];
        for (int i = 0; i < 100000; i++) {
            int id = i;
            threads[i] = Thread.startVirtualThread(() -> {
                if (id % 20000 == 0) {
                    System.out.println("Virtual thread " + id + " running");
                }
            });
        }

        for (Thread t : threads) {
            t.join();
        }

        long end = System.currentTimeMillis();
        System.out.println("Completed in " + (end - start) + " ms");
    }
}
```

41. Executor Service and Callable

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class ExecutorServiceDemo {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newFixedThreadPool(3);
        List<Future<Integer>> results = new ArrayList<>();

        for (int i = 1; i <= 5; i++) {
            int num = i;
            Callable<Integer> task = () -> num * num;
            results.add(executor.submit(task));
        }

        for (Future<Integer> result : results) {
            System.out.println("Result: " + result.get());
        }

        executor.shutdown();
    }
}
```